

**UNIVERSITY OF ENGINEERING & TECHNOLOGY
PESHAWAR**

EE-170L — Computer Programming Lab

Laboratory Session 09

Introduction to Functions in C++

Student Name	M Iqbal Khan
Registration No.	BF25NWELE0698
Submitted To	Sir Rizwan
Course	EE-170L — Computer Programming Lab
Session	Lab 09 — Spring 2026
Department	Electrical Engineering (AI)
Institute	UET Peshawar

1. Objective

The objective of this laboratory session is to understand and apply the concept of functions in C++. By the end of this lab, the student should be able to:

- Define and understand the purpose of functions in C++
- Write function prototypes and function definitions
- Pass parameters to functions and return values
- Call functions from the main() function
- Apply functions to solve practical programming problems

2. Introduction / Theory

2.1 What is a Function?

A function is a named, reusable block of code that performs a specific task. Functions allow a programmer to break a large program into smaller, manageable sections. Each function has a name identifier, may accept parameters, and may return a result.

The most common function encountered so far is main(), which serves as the entry point of every C++ program. Library functions such as getch() and clrscr() are also commonly used functions whose implementation is hidden; we simply call them with appropriate parameters.

2.2 Function Prototype

A function prototype declares the function to the compiler before it is defined. It specifies the function's return type, name, and the types of its parameters. The syntax is:

```
returnType functionName(parameterType1, parameterType2, ...);
```

```
// Examples:  
int    main();  
float  divide(int a, int b);  
void    square(float x);
```

2.3 Function Definition

The function definition provides the actual implementation. It includes the return type, function name, parameters with their names, and the body enclosed in curly braces.

2.4 Function Call

A function is executed when it is called from another function (such as main()). The call must match the prototype in terms of the number and types of arguments provided.

3. Lab Tasks and Code

Task 1 — Function Introduction and Declaration

A function named greet() is declared that takes no parameters and returns nothing (void). It displays a custom greeting message when called from main().

```
#include <iostream>  
using namespace std;  
  
void greet();           // Function prototype  
  
int main()  
{  
    greet();           // Function call  
    return 0;  
}
```

```
}

// Function definition
void greet()
{
    cout << "Welcome to EE-170L Computer Programming Lab!" << endl;
    cout << "Have a productive lab session." << endl;
}
```

Expected Output — Task 1

```
Welcome to EE-170L Computer Programming Lab!
Have a productive lab session.
```

Task 2 — Basic Function Declaration and Call

A function named `sayHello()` is declared that takes no parameters and returns nothing. It displays the message "Hello, World!" when called from `main()`.

```
#include <iostream>
using namespace std;

void sayHello();           // Function prototype

int main()
{
    sayHello();           // Function call
    return 0;
}

// Function definition
void sayHello()
{
    cout << "Hello, World!" << endl;
}
```

Expected Output — Task 2

```
Hello, World!
```

Task 3 — Array Input, Sum Function, and Average Calculation

This program takes 10 integer values as input from the user and stores them in a 1D array. A function named `calculateSum()` accepts the array and its size as parameters and returns the sum of all elements. The `main()` function then computes and displays the average.

```
#include <iostream>
using namespace std;

int calculateSum(int arr[], int size); // Function prototype

int main()
{
    const int SIZE = 10;
    int numbers[SIZE];

    // Input 10 integers from user
    cout << "Enter 10 integer values:" << endl;
    for (int i = 0; i < SIZE; i++)
    {
        cout << "  Value [" << i + 1 << "]: ";
        cin >> numbers[i];
    }
}
```

```

// Call calculateSum function
int sum = calculateSum(numbers, SIZE);

// Calculate and display average
float average = (float)sum / SIZE;

cout << "\nSum      = " << sum      << endl;
cout << "Average = " << average << endl;

return 0;
}

// Function definition
int calculateSum(int arr[], int size)
{
    int total = 0;
    for (int i = 0; i < size; i++)
    {
        total += arr[i];
    }
    return total;
}

```

Sample Output — Task 3

Enter 10 integer values:

```

Value [1]:  5
Value [2]: 10
Value [3]: 15
Value [4]: 20
Value [5]: 25
Value [6]: 30
Value [7]: 35
Value [8]: 40
Value [9]: 45
Value [10]: 50

```

```

Sum      = 275
Average = 27.5

```

4. Analysis and Discussion

4.1 Role of Function Prototypes

The function prototype informs the compiler of the function's return type and parameters before its actual definition appears in the code. This allows functions to be called in `main()` even if their definitions appear later in the file, avoiding compilation errors.

4.2 void vs. int Return Types

Tasks 1 and 2 use void functions because they do not need to return any computed value to the caller — they simply output text. Task 3 uses an int return type for `calculateSum()` because the computed sum must be passed back to `main()` for further use in calculating the average.

4.3 Passing Arrays to Functions

In Task 3, the array is passed to `calculateSum()` along with its size. In C++, arrays are passed by reference (the address of the first element is shared), so the function directly accesses the same data. The size parameter is necessary because C++ does not automatically track array length inside functions.

4.4 Type Casting for Average

Since both `sum` and `SIZE` are integers, dividing them directly would yield an integer result, truncating the decimal portion. The explicit cast `(float)sum` ensures the division is performed in floating-point arithmetic, producing a correct decimal average.

5. Conclusion

This lab provided practical experience with one of the most fundamental building blocks of structured C++ programming — functions. The following key concepts were demonstrated and applied:

- Functions promote code reuse and modularity, reducing repetition in programs.
- Function prototypes must be declared before the function is called so the compiler can verify correct usage.
- void functions are used when no return value is needed; typed functions (`int`, `float`, etc.) return a computed result.
- Parameters allow functions to receive external data and operate on it independently.
- Arrays can be passed to functions along with a size parameter for flexible processing.

By completing all three tasks, the objectives of Lab 09 were successfully achieved.

Student Name: M Iqbal Khan
Registration No.: BF25NWELE0698
Submitted To: Sir Rezwan
Date: Spring 2026